

M18 - Project Evidence Workbook

EXPLAINED WORKBOOK - NOT CHECKBOXES WITHOUT CONTEXT

How to use this

For each project, write short answers and paste/save the actual path or output you created. The prompts explain what the teacher expects. A blank is not asking you to guess a hidden answer; it is asking you to record YOUR project decision/evidence.

P1 - Reliable Batch Pipeline

CSV + paginated API + PostgreSQL

A. Business understanding

Write what the business needs in plain language before tools. Example shape: "Operations needs daily trusted order totals and must not double count redelivered files."

Your notes

- ___ Primary user/team:
- ___ Business result they need:
- ___ Freshness/latency need:
- ___ What wrong result would hurt them most:

B. Source profile and contract

For every source, record what one row/object means and the business key. This prevents later joins/dedup from being guesswork.

Source	One row/object means	Business key	Required fields	Timestamp/late risk	Duplicate/redelivery meaning

C. Trusted output grain

Complete this exact sentence: "One row in _____ represents _____." Then list the key columns that make that row unique under your business rule.

Grain and keys

- ___ Trusted table/output name:
- ___ One row represents:
- ___ Business/natural key:
- ___ Surrogate key if used and why:
- ___ Measure(s) that may be summed safely:

D. Architecture decision

Do not only draw boxes. Explain responsibility and failure behavior.

Layer/box	Input	Persisted output	Trusted?	If it fails...
Source				
Raw/Landing				
Validation/Quarantine				
Trusted/Silver				
Gold/Serving				
Orchestration				

E. Quality and reconciliation

For each critical rule, state what happens to a failing row/batch. Critical means the trusted result must not silently publish as correct.

Rule	Why it matters	Failure action	Evidence path/result
required key			
duplicate/business key			
type/range/date			
relationship			
source-to-trusted reconciliation			

F. Rerun / incremental / late / backfill

These four ideas are related but not identical. Record each separately.

Recovery design

- ___ Rerun idempotency key/rule:
- ___ Incremental watermark/filter:
- ___ Late data policy:
- ___ Backfill command/range strategy:
- ___ How you prove later/newer state is not corrupted:

G. Failure diary

Save the first failure. Root cause is the reason the system behaved incorrectly, not the last line of the error alone.

Incident	First symptom/evidence	Root cause	Smallest repair	Post-repair proof

H. Reproducibility/publication

Final evidence

- ___ Clean-run/clone path:
- ___ Test/checker result:
- ___ Final trusted controls:
- ___ README run command:
- ___ Known limitation:
- ___ One 10x scale change:

I. Defense rehearsal

Answer without reading code. If you need the code open to remember the basic grain/key/failure story, practise again.

60-second defense

___ Problem:

___ Architecture:

___ Grain/key:

___ Quality/reconciliation:

___ Rerun/backfill:

___ Failure + repair:

___ Limitation:

P2 - Analytics Engineering Platform

dbt + Airflow + Docker

A. Business understanding

Write what the business needs in plain language before tools. Example shape: "Operations needs daily trusted order totals and must not double count redelivered files."

Your notes

- ___ Primary user/team:
- ___ Business result they need:
- ___ Freshness/latency need:
- ___ What wrong result would hurt them most:

B. Source profile and contract

For every source, record what one row/object means and the business key. This prevents later joins/dedup from being guesswork.

Source	One row/object means	Business key	Required fields	Timestamp/late risk	Duplicate/redelivery meaning

C. Trusted output grain

Complete this exact sentence: "One row in _____ represents _____." Then list the key columns that make that row unique under your business rule.

Grain and keys

- ___ Trusted table/output name:
- ___ One row represents:
- ___ Business/natural key:
- ___ Surrogate key if used and why:
- ___ Measure(s) that may be summed safely:

D. Architecture decision

Do not only draw boxes. Explain responsibility and failure behavior.

Layer/box	Input	Persisted output	Trusted?	If it fails...
Source				
Raw/Landing				
Validation/Quarantine				
Trusted/Silver				
Gold/Serving				
Orchestration				

E. Quality and reconciliation

For each critical rule, state what happens to a failing row/batch. Critical means the trusted result must not silently publish as correct.

Rule	Why it matters	Failure action	Evidence path/result
required key			
duplicate/business key			
type/range/date			
relationship			
source-to-trusted reconciliation			

F. Rerun / incremental / late / backfill

These four ideas are related but not identical. Record each separately.

Recovery design

- ___ Rerun idempotency key/rule:
- ___ Incremental watermark/filter:
- ___ Late data policy:
- ___ Backfill command/range strategy:
- ___ How you prove later/newer state is not corrupted:

G. Failure diary

Save the first failure. Root cause is the reason the system behaved incorrectly, not the last line of the error alone.

Incident	First symptom/evidence	Root cause	Smallest repair	Post-repair proof

H. Reproducibility/publication

Final evidence

- ___ Clean-run/clone path:
- ___ Test/checker result:
- ___ Final trusted controls:
- ___ README run command:
- ___ Known limitation:
- ___ One 10x scale change:

I. Defense rehearsal

Answer without reading code. If you need the code open to remember the basic grain/key/failure story, practise again.

60-second defense

___ Problem:

___ Architecture:

___ Grain/key:

___ Quality/reconciliation:

___ Rerun/backfill:

___ Failure + repair:

___ Limitation:

P3 - Scalable Lakehouse Pipeline

PySpark + Parquet + Delta + Fabric

A. Business understanding

Write what the business needs in plain language before tools. Example shape: "Operations needs daily trusted order totals and must not double count redelivered files."

Your notes

___ Primary user/team:

___ Business result they need:

___ Freshness/latency need:

___ What wrong result would hurt them most:

B. Source profile and contract

For every source, record what one row/object means and the business key. This prevents later joins/dedup from being guesswork.

Source	One row/object means	Business key	Required fields	Timestamp/late risk	Duplicate/redelivery meaning

C. Trusted output grain

Complete this exact sentence: "One row in _____ represents _____." Then list the key columns that make that row unique under your business rule.

Grain and keys

___ Trusted table/output name:

___ One row represents:

___ Business/natural key:

___ Surrogate key if used and why:

___ Measure(s) that may be summed safely:

D. Architecture decision

Do not only draw boxes. Explain responsibility and failure behavior.

Layer/box	Input	Persisted output	Trusted?	If it fails...
Source				
Raw/Landing				
Validation/Quarantine				
Trusted/Silver				
Gold/Serving				
Orchestration				

E. Quality and reconciliation

For each critical rule, state what happens to a failing row/batch. Critical means the trusted result must not silently publish as correct.

Rule	Why it matters	Failure action	Evidence path/result
required key			
duplicate/business key			
type/range/date			
relationship			
source-to-trusted reconciliation			

F. Rerun / incremental / late / backfill

These four ideas are related but not identical. Record each separately.

Recovery design

- ___ Rerun idempotency key/rule:
- ___ Incremental watermark/filter:
- ___ Late data policy:
- ___ Backfill command/range strategy:
- ___ How you prove later/newer state is not corrupted:

G. Failure diary

Save the first failure. Root cause is the reason the system behaved incorrectly, not the last line of the error alone.

Incident	First symptom/evidence	Root cause	Smallest repair	Post-repair proof

H. Reproducibility/publication

Final evidence

- ___ Clean-run/clone path:
- ___ Test/checker result:
- ___ Final trusted controls:
- ___ README run command:
- ___ Known limitation:
- ___ One 10x scale change:

I. Defense rehearsal

Answer without reading code. If you need the code open to remember the basic grain/key/failure story, practise again.

60-second defense

___ Problem:

___ Architecture:

___ Grain/key:

___ Quality/reconciliation:

___ Rerun/backfill:

___ Failure + repair:

___ Limitation:

CAPSTONE - Marketplace Production Data Platform

multi-source + dbt/Airflow/Spark/Fabric

A. Business understanding

Write what the business needs in plain language before tools. Example shape: "Operations needs daily trusted order totals and must not double count redelivered files."

Your notes

- ___ Primary user/team:
- ___ Business result they need:
- ___ Freshness/latency need:
- ___ What wrong result would hurt them most:

B. Source profile and contract

For every source, record what one row/object means and the business key. This prevents later joins/dedup from being guesswork.

Source	One row/object means	Business key	Required fields	Timestamp/late risk	Duplicate/redelivery meaning

C. Trusted output grain

Complete this exact sentence: "One row in _____ represents _____." Then list the key columns that make that row unique under your business rule.

Grain and keys

- ___ Trusted table/output name:
- ___ One row represents:
- ___ Business/natural key:
- ___ Surrogate key if used and why:
- ___ Measure(s) that may be summed safely:

D. Architecture decision

Do not only draw boxes. Explain responsibility and failure behavior.

Layer/box	Input	Persisted output	Trusted?	If it fails...
Source				
Raw/Landing				
Validation/Quarantine				
Trusted/Silver				
Gold/Serving				
Orchestration				

E. Quality and reconciliation

For each critical rule, state what happens to a failing row/batch. Critical means the trusted result must not silently publish as correct.

Rule	Why it matters	Failure action	Evidence path/result
required key			
duplicate/business key			
type/range/date			
relationship			
source-to-trusted reconciliation			

F. Rerun / incremental / late / backfill

These four ideas are related but not identical. Record each separately.

Recovery design

- ___ Rerun idempotency key/rule:
- ___ Incremental watermark/filter:
- ___ Late data policy:
- ___ Backfill command/range strategy:
- ___ How you prove later/newer state is not corrupted:

G. Failure diary

Save the first failure. Root cause is the reason the system behaved incorrectly, not the last line of the error alone.

Incident	First symptom/evidence	Root cause	Smallest repair	Post-repair proof

H. Reproducibility/publication

Final evidence

- ___ Clean-run/clone path:
- ___ Test/checker result:
- ___ Final trusted controls:
- ___ README run command:
- ___ Known limitation:
- ___ One 10x scale change:

I. Defense rehearsal

Answer without reading code. If you need the code open to remember the basic grain/key/failure story, practise again.

60-second defense

___ Problem:

___ Architecture:

___ Grain/key:

___ Quality/reconciliation:

___ Rerun/backfill:

___ Failure + repair:

___ Limitation: